Ex 8: Deploying a Fine-Tuned Language Model for Customer Support with Evaluation (BLEU & ROUGE)

Learning Objective:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

Steps:

1. **Import Required Libraries**
   Import Transformers, Datasets, Torch, and evaluation libraries (BLEU, ROUGE).
2. **Load Dataset**
   Use a Customer Service dataset or Bank FAQ dataset containing queries and responses.
3. **Data Preprocessing**
   - Clean and normalize text
   - Prepare input-output pairs (query → response)
4. **Split Dataset**
   Divide into training and testing datasets.
5. **Load Pre-trained Model**
   Load models such as T5 or BART from Hugging Face.
6. **Fine-Tune the Model**
   Train the model using the training dataset.
7. **Save the Fine-Tuned Model**
   Store trained model and tokenizer.
8. **Build Inference Pipeline**
   Create a function to generate responses for input queries.
9. **Generate Predictions on Test Dataset**
   - Pass all test queries to the model
   - Generate responses for each query
10. **Compute BLEU Scores (BLEU-1 to BLEU-4)**

- Measure n-gram precision between generated and reference responses
- Compute BLEU-1, BLEU-2, BLEU-3, and BLEU-4 for each sample

11. **Compute ROUGE Scores**

- ROUGE-1 → unigram recall
- ROUGE-2 → bigram recall
- ROUGE-L for sequence similarity

12. **Aggregate Evaluation Metrics**

- Calculate average scores across all test samples
- Prepare overall performance summary

13. **Display Evaluation Results**

Metric	**Score**
BLEU-1	
BLEU-2	
BLEU-3	
BLEU-4	
ROUGE-1	
ROUGE-2	
ROUGE-L	

14. **Develop User Interface**

- Use Streamlit for web app
- OR deploy using Hugging Face Spaces

15. **Deploy the Application**

- Host the model for real-time customer query handling

16. **Test with Sample Queries**
    Validate the deployed system with real-time user inputs.

17. **Display Output**
    Show generated responses in the interface.

Program:

```
# ============================================================
# IMPORT
# ============================================================
import pandas as pd
from sklearn.model_selection import train_test_split
from transformers import T5Tokenizer, T5ForConditionalGeneration
from transformers import Trainer, TrainingArguments
from datasets import Dataset
import evaluate

# ============================================================
# STEP 1: LOAD DATASET FROM INTERNET CSV
# Customer Support Dataset
# ============================================================

url = "https://raw.githubusercontent.com/clinc/oos-eval/master/data/data_full.json"

import json, requests

data = requests.get(url).json()

questions = []
answers = []

# Build Question Answer pairs
for item in data["train"][:2000]:
    questions.append(item[0])
```

```
    answers.append(item[1])

df = pd.DataFrame({
    "question": questions,
    "answer": answers
})

print(df.head())

# ============================================================
# STEP 2: TRAIN TEST SPLIT
# ============================================================
train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42
)

train_dataset = Dataset.from_pandas(train_df)
test_dataset  = Dataset.from_pandas(test_df)

print("Train:", len(train_df))
print("Test :", len(test_df))

# ============================================================
# STEP 3: LOAD MODEL
# ============================================================
model_name = "google/flan-t5-base"
```

```
tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)

# ============================================================
# STEP 4: TOKENIZE
# ============================================================
def preprocess(example):

    inputs = tokenizer(
        example["question"],
        padding="max_length",
        truncation=True,
        max_length=64
    )

    targets = tokenizer(
        example["answer"],
        padding="max_length",
        truncation=True,
        max_length=32
    )

    inputs["labels"] = targets["input_ids"]

    return inputs

train_tokenized = train_dataset.map(preprocess)
```

```
test_tokenized = test_dataset.map(preprocess)

# ============================================================
# STEP 5: TRAIN
# ============================================================
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=2,
    per_device_train_batch_size=4,
    logging_steps=50,
    save_strategy="no"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized
)

trainer.train()

# Save
model.save_pretrained("customer_model")
tokenizer.save_pretrained("customer_model")
import torch

# Detect device
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
# Move model to same device
model = model.to(device)

predictions = []
references = []

for q, a in zip(test_df["question"], test_df["answer"]):

    # Tokenize and move input to device
    inputs = tokenizer(
        q,
        return_tensors="pt",
        truncation=True,
        padding=True
    )

    input_ids = inputs.input_ids.to(device)
    attention_mask = inputs.attention_mask.to(device)

    # Generate prediction
    output = model.generate(
        input_ids=input_ids,
        attention_mask=attention_mask,
        max_length=20
    )

    pred = tokenizer.decode(
```

```
        output[0],
        skip_special_tokens=True
    )

    predictions.append(pred)
    references.append([a])

print("Prediction Completed Successfully")
# ============================================================
# STEP 7: SAVE CSV
# ============================================================
result = pd.DataFrame({
    "Question": test_df["question"].values,
    "Actual": test_df["answer"].values,
    "Predicted": predictions
})

result.to_csv("predictions.csv", index=False)

print("Saved predictions.csv")

# ============================================================
# STEP 8: EVALUATE
# ============================================================
rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")

print("ROUGE:",
```

```
    rouge.compute(
        predictions=predictions,
        references=test_df["answer"].tolist()
    ))

print("BLEU:",
    bleu.compute(
        predictions=predictions,
        references=references
    ))

# ============================================================
# STEP 9: STREAMLIT APP
# ============================================================
app_code = '''
import streamlit as st
from transformers import T5Tokenizer, T5ForConditionalGeneration

model = T5ForConditionalGeneration.from_pretrained("customer_model")
tokenizer = T5Tokenizer.from_pretrained("customer_model")

st.title("Customer Support Chatbot")

query = st.text_input("Ask Question")

if query:
    ids = tokenizer(query, return_tensors="pt").input_ids
    output = model.generate(ids, max_length=20)
```

```
    ans = tokenizer.decode(output[0], skip_special_tokens=True)
    st.success(ans)
'''

with open("app.py", "w") as f:
    f.write(app_code)

# ============================================================
# STEP 10: RUN STREAMLIT
# ============================================================
!streamlit run app.py &>/content/logs.txt &

# ============================================================
# STEP 11: PUBLIC URL
# ============================================================
!npm install -g localtunnel

!lt --port 8501
```

Learning Outcome:

Students will be able to fine-tune, evaluate, and deploy a language model for customer support systems. They will gain practical experience in computing **BLEU and ROUGE scores**, enabling proper performance assessment of NLP models in real-world applications.